ITS Villada

Base de Datos II

INFORME DE INVESTIGACIÓN

Grupo N° 8

Procesamiento distribuido y *machine learning* sobre datos masivos con Apache Spark MLlib

Arquitectura, Algoritmos Iterativos y Optimización a Gran Escala

Integrantes

Leon Guchea
Bautista Daveloze
Riccioni Santiago
Sibona Luciano
Macias Luciano

Docentes

Lisandro Márquez
Teodoro Reyna

11 de noviembre de 2026

***Resumen*—Cuando el volumen de un conjunto de datos excede las capacidades de almacenamiento y cómputo de un único servidor, su análisis exige estrategias de particionamiento horizontal y procesamiento paralelo en clúster. Este informe examina los principios arquitectónicos y algorítmicos del procesamiento distribuido moderno, analizando la transición desde el paradigma MapReduce hacia el modelo en memoria de Apache Spark, sus RDD, DataFrames y el optimizador Catalyst. Se profundiza en el funcionamiento de Spark MLlib para el aprendizaje automático sobre datos masivos, adoptando el algoritmo de agrupamiento K-Means como caso de estudio representativo de algoritmos iterativos. Se analizan formalmente la paralelización mediante estadísticos suficientes, la agregación jerárquica en árbol (*treeAggregate*), las canalizaciones (*ML Pipelines*), los mecanismos de tolerancia a fallos por linaje y la mitigación de tareas rezagadas mediante ejecución especulativa. Asimismo, se contrasta el procesamiento analítico distribuido con las bases de datos relacionales tradicionales y se detallan las técnicas críticas de optimización del motor (Project Tungsten, gestión de memoria fuera de la JVM) y la resolución de cuellos de botella por sesgo de datos (*data skew*) y operaciones de *shuffle*.**

***Palabras clave*—Apache Spark, MLlib, MapReduce, Hadoop, K-Means, procesamiento distribuido, RDD, tolerancia a fallos, big data, Catalyst, Project Tungsten, treeAggregate.**

I. Introducción

El volumen de datos que generan hoy las organizaciones (registros transaccionales, eventos de aplicaciones, telemetría de sensores y flujos de interacción en tiempo real) crece a un ritmo superior a la capacidad de una única máquina para almacenarlos y procesarlos. Ante este límite físico existen dos caminos: escalar verticalmente (*scale-up*), adquiriendo servidores con mayor cantidad de núcleos y memoria, o escalar horizontalmente (*scale-out*), distribuyendo el conjunto de datos y la carga de cómputo entre múltiples computadoras conectadas en red. La vía horizontal resulta económicamente viable y no posee un techo rígido de hardware, pero introduce retos complejos de sistemas distribuidos: división coherente de datos, coordinación inter-nodo, sobrecarga en la red (*network traffic*) y preservación de consistencia ante fallos parciales de hardware.

Google formalizó una primera respuesta masiva a estos problemas con el modelo MapReduce [1], que el ecosistema Apache popularizó a través de Hadoop y su sistema de archivos distribuido HDFS [2]. Si bien MapReduce resultó altamente robusto y eficaz para transformaciones por lotes de una sola pasada, evidenció severas ineficiencias en algoritmos iterativos característicos del aprendizaje automático (*Machine Learning*), los cuales requieren recorrer las mismas estructuras de datos decenas de veces. Apache Spark [3], [4] surgió específicamente para resolver esta deficiencia, conservando los datos intermedios en memoria entre iteraciones y garantizando tolerancia a fallos sin recurrir a la replicación en disco.

Este informe tiene dos objetivos principales:

1) Analizar los fundamentos teóricos y la arquitectura de Apache Spark y de su biblioteca de aprendizaje automático, MLlib [6], contrastando su paradigma distribuido y analítico con el procesamiento de las bases de datos relacionales tradicionales.
2) Estudiar en profundidad la paralelización de algoritmos iterativos sobre macrodatos, tomando como núcleo analítico a K-Means, y examinando los patrones de agregación jerárquica (*treeAggregate*), las canalizaciones (*ML Pipelines*), las optimizaciones de bajo nivel del motor (Catalyst y Project Tungsten) y las estrategias para resolver cuellos de botella en producción.

El resto del informe se estructura de la siguiente manera: la Sección II expone el marco teórico del procesamiento distribuido y la arquitectura de Spark; la Sección III examina los casos de uso y compara Spark con los sistemas relacionales; la Sección IV formaliza el algoritmo K-Means y su paralelización mediante estadísticos suficientes; la Sección V analiza la tolerancia a fallos por linaje, la ejecución especulativa y la comparación temporal analítica con Hadoop; la Sección VI profundiza en el aprendizaje automático a escala masiva, agregación en árbol, canalizaciones, optimizaciones del motor y mitigación de cuellos de botella; finalmente, la Sección VII resume las conclusiones.

II. Marco teórico

A. El modelo MapReduce

MapReduce [1] estructura cualquier cómputo distribuido sobre dos funciones parametrizadas por el desarrollador. La función *map* procesa de manera aislada y paralela cada fragmento de la entrada, emitiendo pares clave–valor intermedios. Posteriormente, el entorno de ejecución agrupa todos los valores asociados a una misma clave y los distribuye a través de la red en una fase denominada *shuffle*. Finalmente, la función *reduce* procesa la lista agregada de valores para cada clave, generando la salida consolidada. De manera complementaria, una función local *combiner* puede aplicar una reducción previa en cada nodo antes del *shuffle*, disminuyendo drásticamente el tráfico cuando la operación es asociativa y conmutativa.

El sistema subyacente se encarga de particionar la entrada, planificar tareas, reintentar ejecuciones fallidas y supervisar la red. Para asegurar resiliencia absoluta, cada etapa de MapReduce lee sus entradas desde HDFS y almacena sus salidas en disco con replicación triple [2]. Si bien este mecanismo garantiza alta tolerancia a fallos, impone una penalización de rendimiento prohibitiva en algoritmos iterativos: cada paso sucesivo debe encadenar un trabajo independiente que paga el costo de escribir y volver a leer del disco todo el conjunto de datos.

B. Apache Spark y los RDD

Spark [3] supera esta barrera mediante el concepto de *Resilient Distributed Dataset* (RDD) [4]: una colección distribuida, inmutable y particionada de objetos en memoria. Sobre los RDD se definen dos categorías de operaciones:

- **Transformaciones** (`map`, `filter`, `groupByKey`, ...): Definen un nuevo RDD aplicando una función sobre uno existente. Son de evaluación perezosa (*lazy evaluation*): no se calculan al invocarse, sino que se ensamblan en un grafo dirigido acíclico (DAG) que describe el linaje lógico de cómputo.

- **Acciones** (`count`, `collect`, `reduce`, ...): Requieren materializar un resultado hacia el *driver* o escribirlo en almacenamiento, lo que desencadena la optimización y ejecución efectiva del DAG.

La evaluación perezosa permite que el planificador inspeccione el grafo de ejecución completo antes de despachar tareas. De este modo, Spark clasifica las dependencias en dos tipos:

1) *Dependencias estrechas* (*narrow*): Cada partición del RDD padre es utilizada por a lo sumo una partición del RDD hijo (ej. `map`, `filter`). Permiten ejecutar múltiples operaciones encadenadas en memoria dentro del mismo hilo (*pipelining*) sin sincronización por red.
2) *Dependencias amplias* (*wide*): Múltiples particiones hijas dependen de datos distribuidos en distintas particiones padres (ej. `reduceByKey`, `join`). Requieren una costosa etapa de *shuffle* y delimitan las fronteras entre etapas (*stages*) de ejecución.

Dos propiedades diferencian decisivamente a los RDD. La primera es la **persistencia explícita**: mediante `cache()` o `persist()`, el programador puede mantener un RDD materializado en la memoria RAM de los nodos trabajadores tras su primera evaluación, permitiendo su reutilización en decenas de iteraciones sin incurrir en I/O de almacenamiento. La segunda es el **linaje** (*lineage*): en lugar de mantener copias redundantes de conjuntos intermedios en disco, cada partición retiene la fórmula exacta de transformaciones que la originaron. Ante la pérdida de un nodo, Spark simplemente reejecuta las transformaciones sobre la partición afectada desde la fuente persistente [4].

C. Arquitectura de ejecución

Una aplicación en Apache Spark se compone de cuatro entidades estructurales [12], representadas esquemáticamente en la Fig. 1:

- **Driver**: Proceso principal que ejecuta la función `main` del programa. Traduce el código a un grafo DAG, descompone el plan en etapas y tareas, programa su ejecución y consolida resultados finales.
- **Cluster Manager** (o *master*): Administra la asignación de recursos computacionales físicos o virtualizados en el clúster. Puede operar en modo nativo (*Standalone*), YARN [7] o Kubernetes.
- **Worker Node**: Servidor físico o máquina virtual perteneciente al clúster, responsable de alojar y mantener procesos ejecutores.
- **Executor**: Proceso independiente instanciado en un *worker* para una aplicación específica. Ejecuta tareas concurrentes en hilos sobre particiones de datos y conserva fragmentos de RDD persistidos en su espacio de memoria.

D. Spark MLlib

MLlib es el componente nativo de Apache Spark destinado al aprendizaje automático escalable [6]. Provee implementaciones distribuidas de algoritmos de agrupamiento, clasificación, regresión, análisis de componentes principales y recomendación colaborativa. Su ventaja arquitectónica radica en que el mismo código ensayado en un entorno local puede escalarse a miles de núcleos y terabytes de datos sin modificaciones algorítmicas, aprovechando la infraestructura de paralelismo de datos.

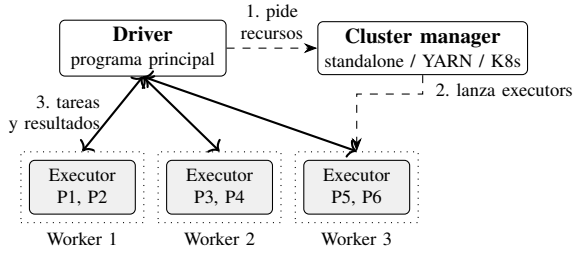


Fig. 1. Arquitectura de ejecución en Apache Spark. Tras la asignación de recursos por el Cluster Manager, el Driver despacha tareas directamente a los Executors y recolecta resultados.

III. Casos de uso y comparación con el modelo relacional

A. Escenarios de aplicación y cargas de trabajo

Spark está diseñado para cargas analíticas masivas (OLAP, *Online Analytical Processing*) y procesamiento por lotes sobre volúmenes que desbordan un servidor individual. Un escenario representativo es un flujo ETL (*Extract, Transform, Load*) que ingiere terabytes de eventos web o registros transaccionales, aplica desnormalizaciones y agrega métricas para consultas de inteligencia de negocio mediante Spark SQL [14]. Otro escenario típico es el entrenamiento distribuido de modelos predictivos y de segmentación con MLlib sobre lagos de datos (*data lakes*). El caso de estudio de K-Means analizado en este trabajo modela la segmentación masiva de usuarios o transacciones a partir de datos consolidados en lagos de datos empresariales.

Por el contrario, Spark no constituye un reemplazo adecuado para motores de bases de datos relacionales en entornos operacionales y transaccionales (OLTP, *Online Transaction Processing*). En escenarios donde se requieren lecturas y modificaciones puntuales de registros individuales con latencias de milisegundos, concurrencia de miles de clientes concurrentes y restricciones estrictas de integridad referencial, los motores relacionales clásicos superan ampliamente a los sistemas distribuidos basados en lotes. Distribuir un conjunto de datos pequeño en un clúster Spark introduce sobrecargas de serialización, red y coordinación que degradan el rendimiento en comparación con una ejecución mono-nodo optimizada.

B. Diferencias y complementariedad arquitectónica

Spark no es una base de datos: carece de almacenamiento propio dependiente de disco y de mecanismos nativos de transacciones ACID para escrituras atómicas concurrentes a nivel de tupla. Aunque Spark SQL ofrece una sintaxis declarativa idéntica al estándar relacional, las consultas se traducen en planes de transformaciones distribuidas sobre particiones completas de datos [14].

TABLA I
COMPARACIÓN ENTRE BASES DE DATOS RELACIONALES Y APACHE SPARK

Dimensión	**Base de datos relacional**	**Apache Spark**
Propósito central	Transacciones OLTP y consultas operacionales	Procesamiento analítico OLAP y ML distribuido
Unidad operativa	Filas, índices B-Tree y tablas relacionales	Particiones distribuidas, RDD y DataFrames
Patrón de acceso	Lecturas/escrituras atómicas puntuales indexadas	Escaneo masivo, transformaciones y reducciones
Escalabilidad	Vertical predominante (o sharding complejo)	Horizontal elástica sobre clústeres de cómputo
Garantías ACID	Integridad referencial y transacciones nativas	Dependientes del sistema de almacenamiento destino
Uso de SQL	Consultas sobre esquemas fijos indexados	Optimización de planes masivos con Catalyst

Asimismo, las bases de datos relacionales tradicionales preservan la persistencia e integridad mediante registros de escritura previa (*Write-Ahead Logging*, WAL) y réplicas de almacenamiento [17]. Spark, en contraposición, recurre al linaje determinista del DAG para reconstruir particiones perdidas en memoria, delegando la persistencia duradera final al sistema de archivos subyacente (ej. HDFS, Amazon S3).

En la práctica empresarial ambas tecnologías son complementarias: los sistemas relacionales registran el tráfico transaccional continuo y periódicamente vuelcan sus datos a almacenes distribuidos, donde clústeres de Spark ejecutan procesos de agregación, minería y entrenamiento analítico.

IV. K-MEANS DISTRIBUIDO

A. Formulación del algoritmo

Dado un conjunto de n puntos multidimensionales $x_1, \ldots, x_n \in \mathbb{R}^d$ y un número predeterminado k de conglomerados, K-Means busca hallar un conjunto de centroides $\mu_1, \ldots, \mu_k \in \mathbb{R}^d$ que minimice la función de costo de error cuadrático intra-grupo (WSSSE, *Within-Set Sum of Squared Errors*):

$$J = \sum_{i=1}^{n} \min_{j \in \{1,\ldots,k\}} \|x_i - \mu_j\|^2. \tag{1}$$

El algoritmo clásico de Lloyd [8], [9] aproxima numéricamente la solución alternando iterativamente dos fases hasta alcanzar la convergencia:

1) **Asignación**: Cada punto x_i se asocia al centroide más próximo según la distancia euclídea:

$$c(i) = \arg \min_{j \in \{1,\ldots,k\}} \|x_i - \mu_j\|^2. \tag{2}$$

2) **Actualización**: Se recalculan los k centroides como el baricentro de todos los puntos asignados a cada grupo j:

$$\mu_j = \frac{1}{|C_j|} \sum_{i \in C_j} x_i, \quad \text{donde } C_j = \{i : c(i) = j\}. \tag{3}$$

Dado que cada iteración reduce monotónicamente o conserva el costo J, el algoritmo converge en un número finito de pasos a un mínimo local condicionado por los centroides iniciales.

B. Paralelización mediante estadísticos suficientes

La fase de asignación exhibe un paralelismo elemental (*embarrassingly parallel*): conociendo los k centroides actuales, cualquier ejecutor puede clasificar sus puntos asignados de manera estrictamente local y sin consultar a otros nodos. En contraste, la fase de actualización requiere consolidar la información global de todas las particiones del clúster.

Transmitir todos los puntos asignados al *driver* anularía las ventajas del procesamiento distribuido. Para evitarlo, cada partición local p calcula para cada grupo j un vector de suma acumulada $S_{p,j} \in \mathbb{R}^d$ y un contador escalar $N_{p,j} \in \mathbb{N}$:

$$S_{p,j} = \sum_{i \in p,\, c(i)=j} x_i, \qquad N_{p,j} = \big|\{i \in p : c(i) = j\}\big|. \tag{4}$$

El par (S, N) constituye un **estadístico suficiente**: encapsula toda la información requerida para computar el promedio global exacto. Como la suma vectorial y la suma escalar son operaciones asociativas y conmutativas, pueden reducirse en cualquier orden a lo largo del clúster. Una vez acumuladas las sumas globales, el nuevo centroide se obtiene mediante una única división:

$$\mu'_j = \frac{\sum_p S_{p,j}}{\sum_p N_{p,j}}. \tag{5}$$

El volumen de datos a transferir por partición se reduce a $k(d+1)$ números en punto flotante, magnitud completamente independiente del número n de registros.

C. El sesgo del promedio de promedios

Una aproximación errónea recurrente consiste en calcular directamente en cada partición el vector promedio local $\bar{x}_{p,j} = S_{p,j}/N_{p,j}$ y promediar posteriormente dichos vectores en el *driver*. Esta estrategia produce un resultado matemáticamente incorrecto cuando las particiones poseen cardinalidades desiguales en un conglomerado, al ignorar la ponderación $N_{p,j}$. Considérese el caso unidimensional con un punto en $x = 10$ en la partición 1 y tres puntos en $x = 0$ en la partición 2:

$$\underbrace{\frac{10+0}{2} = 5}_{\text{Promedio de promedios (erróneo)}} \neq \underbrace{\frac{10+0+0+0}{4} = 2{,}5}_{\text{Promedio real ponderado}}. \tag{6}$$

La Tabla II compara tres posibles estrategias de comunicación entre los *executors* y el *driver* durante cada iteración, analizadas para un escenario con $k = 3$, $d = 2$ y números en punto flotante de 8 bytes (doble precisión).

La opción A es correcta pero traslada todos los registros individuales al *driver*, anulando el beneficio de la computación distribuida y saturando el ancho de banda de red. La opción B reduce drásticamente el tráfico pero introduce un sesgo matemático inaceptable, ya que no pondera las cardinalidades de cada partición. La opción C es mínima en volumen de red ($O(kd)$ independiente de n_p) y matemáticamente exacta,

TABLA II
ESTRATEGIAS DE COMUNICACIÓN ENTRE EXECUTORS Y DRIVER

Estrategia	Contenido	Volumen	Exacto
A (Centralizada)	Puntos completos de partición	$\propto n_p$ (19 MiB/10^6)	Sí
B (Promedio local)	Promedio local $\bar{x}_{p,j}$	48 B	No
C (Estadístico suf.)	Suma $S_{p,j}$ y conteo $N_{p,j}$	72 B	Sí

aplicando el concepto del *combiner* de MapReduce al agrupamiento distribuido.

D. Inicialización e implementación en PySpark MLlib

Dado que K-Means es sensible a la posición inicial de los centros, Spark implementa el algoritmo *k-means*$\|$ [10], una generalización paralela de *k-means*++ que muestrea múltiples centroides candidatos simultáneamente en $O(\log n)$ pasadas sobre el clúster. Los parámetros centrales del estimador `pyspark.ml.clustering.KMeans` son la cantidad de grupos `k`, la tolerancia de convergencia `tol`, el límite de iteraciones `maxIter` y la semilla aleatoria `seed` [13]. El Listado 1 exhibe un flujo de entrenamiento básico.

Listado 1. K-Means con PySpark y MLlib

```
from pyspark.ml.clustering import KMeans
from pyspark.ml.feature import VectorAssembler
df = spark.read.csv("clientes.csv", header=True,
     inferSchema=True)
vec = VectorAssembler(inputCols=["x", "y"], outputCol="
     features")
datos = vec.transform(df).cache()
modelo = KMeans(k=3, maxIter=12, seed=1).fit(datos)
print(modelo.clusterCenters())
print(modelo.summary.trainingCost)  # WSSSE
```

La invocación a `cache()` resulta determinante: asegura que las transformaciones se computen una sola vez y residan en la RAM de los ejecutores, acelerando drásticamente las iteraciones sucesivas.

V. TOLERANCIA A FALLOS Y RENDIMIENTO

A. Recuperación por linaje

En clústeres a gran escala compuestos por decenas o cientos de nodos, las fallas transitorias o definitivas de hardware son eventos esperados. Hadoop mitiga este fenómeno replicando bloques de datos tres veces en disco local [2]. Spark, en contrapartida, evita la costosa replicación en memoria de estructuras intermedias apoyándose en el linaje del RDD [4]. Cada RDD mantiene información de solo lectura sobre sus transformaciones ascendentes. Si un nodo ejecutor cae durante la iteración 10 de K-Means, el *driver* detecta la pérdida de la tarea y la reasigna a otro ejecutor disponible, quien reconstruye exclusivamente la partición afectada releyendo su porción de la fuente base persistente.

B. Tareas rezagadas y ejecución especulativa

La velocidad de una etapa paralela está gobernada por su tarea más lenta (*straggler*). Los rezagados surgen por degradación de discos locales, contención de red o competencia de hilos. Dean y Ghemawat propusieron neutralizar este cuello de botella despachando tareas duplicadas de respaldo (*backup tasks*) [1]. Spark implementa esta estrategia mediante el parámetro `spark.speculation`. Cuando se habilita, el sistema monitorea la duración de las tareas y relanza una copia especulativa en otro nodo si una tarea en ejecución supera un múltiplo configurado (`spark.speculation.multiplier`) de la mediana de su etapa, adoptando los resultados de la primera instancia que finalice con éxito [15].

TABLA III
COMPARACIÓN ENTRE HADOOP MAPREDUCE Y APACHE SPARK

Característica	Hadoop MapReduce	Apache Spark
Datos entre pasos	Disco distribuido (HDFS)	Memoria RAM (`cache`)
Modelo de cómputo	Map y Reduce acoplados	Grafo acíclico (DAG) flexible
Tolerancia a fallos	Replicación física de bloques	Linaje determinista y recálculo
Soporte iterativo	Un trabajo aislado por paso	Un único contexto de ejecución
APIs soportadas	Java	Python, Scala, SQL, R, Java

C. Comparación analítica entre Spark y Hadoop

Sea r el costo temporal de leer una partición desde disco, c el tiempo de cálculo local por iteración y w el tiempo de volcar resultados intermedios a almacenamiento persistente. Para un algoritmo que requiere m iteraciones sucesivas, el costo acumulado de Hadoop MapReduce se modela como:

$$T_{\text{Hadoop}} \approx m\,(r + c + w), \tag{7}$$

mientras que Spark, preservando los datos en memoria tras la primera lectura, requiere:

$$T_{\text{Spark}} \approx r + m\,c. \tag{8}$$

En aplicaciones iterativas donde la I/O domina sobre el cómputo ($r + w \gg c$), la aceleración teórica crece casi proporcionalmente a m. Investigaciones empíricas reportan reducciones de tiempo de hasta un orden de magnitud a favor de Spark en algoritmos de aprendizaje automático [4]. La Tabla III sintetiza las diferencias paradigmáticas.

VI. MACHINE LEARNING A ESCALA: ARQUITECTURA, PIPELINES Y OPTIMIZACIÓN

La paralelización básica descrita en la Sección IV asume una comunicación plana hacia el maestro. Sin embargo, en despliegues con petabytes de datos y miles de particiones, la arquitectura debe resolver problemas de saturación de red, orquestación de características, optimización de hardware y asimetría de particiones.

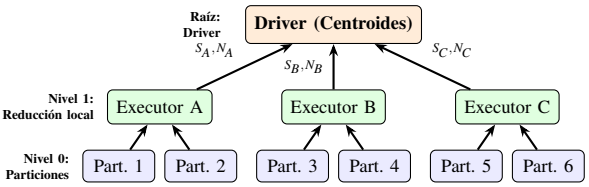


Fig. 2. Agregación jerárquica con `treeAggregate` en Spark MLlib. Los ejecutores intermedios combinan estadísticos suficientes en paralelo antes de transmitirlos al nodo maestro, evitando saturar la interfaz de red del *driver*.

A. Comunicación distribuida y agregación en árbol

Cuando un clúster opera con miles de particiones P, enviar los vectores de estadísticos $(S_{p,j}, N_{p,j})$ directamente al *driver* origina un cuello de botella $O(P)$ de conexiones y tráfico de red entrante sobre un único puerto, desencadenando agotamiento de memoria (*Out Of Memory*, OOM) y serialización forzada de las adiciones.

Para resolver esta vulnerabilidad, Spark MLlib implementa la **agregación jerárquica en árbol** (`treeAggregate` y `treeReduce`) [6]. En lugar de transferir todos los mensajes a un punto central, la reducción se estructura en niveles sucesivos con una profundidad configurable $h \geq 2$ (`depth`), como se ilustra en la Fig. 2.

En el nivel 0, cada ejecutor consolida las estadísticas de sus particiones locales. En los niveles intermedios, un grupo de ejecutores actúa como agregador de un factor de ramificación $b = \lceil P^{1/h} \rceil$ de particiones hijas. Este esquema garantiza que:

- La cantidad de mensajes que ingresan al *driver* disminuye desde $O(P)$ hasta $O(P^{1/h})$.
- El cómputo aritmético de suma vectorial se ejecuta en paralelo entre múltiples núcleos distribuidos.
- La latencia total de agregación escala de manera logarítmica respecto al volumen de particiones.

B. Canalizaciones de Machine Learning (ML Pipelines)

En la práctica industrial de macrodatos, la fase de ajuste del modelo suele representar menos del 20 % del cómputo; el preprocesamiento, escalamiento, imputación y codificación de atributos acaparan el resto de la carga. La API basada en DataFrames (`pyspark.ml`) estandariza estos flujos de trabajo mediante tres abstracciones fundamentales:

- **Transformer**: Algoritmo que recibe un DataFrame y produce un nuevo DataFrame derivado (añadiendo columnas de características, vectores densos o predicciones). Operan aplicando transformaciones fila por fila con dependencias estrechas, procesándose en paralelo local sin originar transferencias por *shuffle*.
- **Estimator**: Algoritmo parametrizado que implementa el método `fit()` sobre un DataFrame para aprender patrones de los datos, produciendo como resultado un *Transformer* entrenado.
- **Pipeline**: Grafo lineal ordenado que encadena múltiples transformadores y estimadores. Garantiza la consistencia del flujo, erradica la fuga de datos (*data leakage*) entre conjuntos de entrenamiento y evaluación y facilita la exportación atómica del modelo a producción.

El Listado 2 muestra una canalización distribuida completa que realiza ensamblado vectorial, escalamiento y entrenamiento de K-Means, junto con su posterior evaluación de calidad mediante el coeficiente Silhouette.

Listado 2. Canalización completa de Machine Learning con PySpark

```
from pyspark.ml import Pipeline
from pyspark.ml.feature import VectorAssembler,
    StandardScaler
from pyspark.ml.clustering import KMeans
from pyspark.ml.evaluation import ClusteringEvaluator

# Ingestion columnar Parquet y preparacion distribuida
df = spark.read.parquet("hdfs:///datos/clientes.parquet")
vec = VectorAssembler(inputCols=["ingreso", "antiguedad", "
    gastos"], outputCol="vec")
std = StandardScaler(inputCol="vec", outputCol="features",
    withMean=True, withStd=True)

# Definicion y ajuste del Pipeline con estimador KMeans
kmeans = KMeans(k=4, initMode="k-means||", maxIter=25, seed
    =42)
pipeline = Pipeline(stages=[vec, std, kmeans])
modelo = pipeline.fit(df)

# Evaluacion distribuida con metrica Silhouette
pred = modelo.transform(df)
score = ClusteringEvaluator(metricName="silhouette").
    evaluate(pred)
print(f"Silhouette␣=␣{score:.4f}")
```

C. El motor de optimización: Catalyst y Project Tungsten

El elevado rendimiento de Spark sobre grandes volúmenes de datos descansa sobre dos componentes internos de su motor relacional y analítico [11]:

Catalyst Optimizer: Motor de optimización extensible que aplica álgebra relacional sobre árboles sintácticos de operaciones en cuatro etapas consecutivas:

1) *Análisis*: Resuelve identificadores de columnas, tipos y esquemas contra el catálogo.
2) *Optimización lógica*: Aplica reglas canónicas deterministas, destacando el empuje de predicados (*predicate pushdown*) que filtra registros directamente en el formato de almacenamiento (ej. omitiendo bloques en archivos Parquet sin cargarlos a memoria) y la poda de proyecciones (*column pruning*) para ignorar columnas redundantes.
3) *Optimización física*: Evalúa planes de ejecución mediante modelos de costo (CBO), seleccionando automáticamente estrategias eficientes como transformar uniones costosas en *Broadcast Hash Joins* cuando una tabla es pequeña, eliminando la etapa de *shuffle*.
4) *Generación de código*: Traduce partes del plan optimizado a *bytecode* nativo de Java para su ejecución inmediata.

Project Tungsten: Rediseño de bajo nivel enfocado en optimizar el cómputo a nivel de CPU y memoria física, eliminando la sobrecarga intrínseca de la máquina virtual Java (JVM):

- *Memoria Off-Heap y formato binario*: Spark gestiona memoria en formato compacto (`UnsafeRow`), eliminando encabezados de objetos Java (16–24 bytes por tupla) y pausas de recolección de basura (*GC pauses*).

- *Whole-Stage Code Generation*: Sintetiza en tiempo de ejecución bucles cerrados en Java plano que compilan cadenas completas de operadores, manteniendo datos en los registros y cachés L1/L2/L3 del procesador.

D. Desafíos operativos: Sesgo de datos y gestión de Shuffle

En entornos productivos, el rendimiento del clúster puede degradarse drásticamente si no se gestionan adecuadamente los cuellos de botella de particionamiento [16]:

Sesgo de datos (*Data Skew*): Se manifiesta cuando una distribución no uniforme en las claves de particionamiento concentra una proporción masiva de registros en un único nodo. El ejecutor afectado se convierte en un cuello de botella que retarda la culminación de toda la etapa paralela. Para contrarrestarlo se aplican dos técnicas:

1) *Salting*: Adición de un prefijo o sufijo aleatorio uniforme $s \in [0, S-1]$ a las claves problemáticas, permitiendo repartir los registros en S subparticiones durante la primera agregación y unificar los resultados mediante una reducción posterior.
2) *Adaptive Query Execution (AQE)*: Característica introducida en versiones recientes que analiza estadísticas en tiempo de ejecución al concluir cada etapa de *shuffle*, subdividiendo automáticamente particiones anómalas sin requerir reescritura manual del código.

Desbordamiento a disco (*Shuffle Spill*): Durante las etapas de *shuffle*, los ejecutores ordenan y reagrupan grandes volúmenes de datos. Si la memoria asignada a la ejecución (`spark.memory.fraction`) es desbordada, Spark escribe temporalmente los fragmentos excedentes a disco local sin compresión (*spill to disk*), penalizando la velocidad entre un factor de 10x y 50x. Se previene calibrando el número de particiones (`spark.sql.shuffle.partitions`, buscando bloques de 100 a 200 MiB) y configurando el serializador binario Kryo (`KryoSerializer`) en sustitución de la serialización nativa de Java.

VII. Conclusiones

El procesamiento distribuido sobre macrodatos constituye una necesidad técnica cuando el volumen, la velocidad y la dimensionalidad de la información desbordan la capacidad de un servidor individual. Apache Spark superó las limitaciones de I/O en disco del modelo MapReduce al introducir colecciones inmutables en memoria con tolerancia a fallos basada en linaje determinista, posibilitando la ejecución eficiente de algoritmos iterativos.

El estudio formal de K-Means demostró que el escalamiento horizontal efectivo depende de una formulación algorítmica adecuada: el uso de estadísticos suficientes reduce la transmisión de red a una magnitud independiente del número de instancias, mientras que la agregación jerárquica en árbol (`treeAggregate`) previene la saturación del nodo maestro al distribuir el cómputo de consolidación.

Asimismo, se evidenció que los sistemas de gestión de bases de datos relacionales y los motores distribuidos como Spark cumplen roles complementarios. Mientras que los primeros ofrecen garantías transaccionales ACID y latencias de milisegundos en operaciones de negocio operacionales (OLTP), Spark provee la infraestructura analítica masiva y elástica (OLAP) requerida para la ingeniería de datos y el entrenamiento de modelos de aprendizaje automático a gran escala.

Finalmente, la consecución de una alta eficiencia en producción exige optimizaciones a nivel de motor de ejecución y control de infraestructura. Los avances provistos por el optimizador Catalyst y Project Tungsten permiten aprovechar al máximo las capacidades físicas del hardware moderno; sin embargo, el diseño adecuado de esquemas de particionamiento, la mitigación del sesgo de datos (*data skew*) y el control del desbordamiento en el *shuffle* siguen siendo responsabilidades cardinales de la ingeniería de datos para garantizar la escalabilidad lineal en el procesamiento masivo.

Referencias

[1] J. Dean and S. Ghemawat, "MapReduce: Simplified data processing on large clusters," in *Proc. 6th USENIX Symp. Operating Systems Design and Implementation (OSDI)*, San Francisco, CA, USA, 2004, pp. 137–150.

[2] K. Shvachko, H. Kuang, S. Radia, and R. Chansler, "The Hadoop distributed file system," in *Proc. IEEE 26th Symp. Mass Storage Systems and Technologies (MSST)*, 2010, pp. 1–10.

[3] M. Zaharia, M. Chowdhury, M. J. Franklin, S. Shenker, and I. Stoica, "Spark: Cluster computing with working sets," in *Proc. 2nd USENIX Workshop Hot Topics in Cloud Computing (HotCloud)*, 2010.

[4] M. Zaharia *et al.*, "Resilient distributed datasets: A fault-tolerant abstraction for in-memory cluster computing," in *Proc. 9th USENIX Symp. Networked Systems Design and Implementation (NSDI)*, San Jose, CA, USA, 2012, pp. 15–28.

[5] M. Zaharia *et al.*, "Apache Spark: A unified engine for big data processing," *Commun. ACM*, vol. 59, no. 11, pp. 56–65, 2016.

[6] X. Meng *et al.*, "MLlib: Machine learning in Apache Spark," *J. Mach. Learn. Res.*, vol. 17, no. 34, pp. 1–7, 2016.

[7] V. K. Vavilapalli *et al.*, "Apache Hadoop YARN: Yet another resource negotiator," in *Proc. 4th ACM Symp. Cloud Computing (SoCC)*, 2013, pp. 5:1–5:16.

[8] S. Lloyd, "Least squares quantization in PCM," *IEEE Trans. Inf. Theory*, vol. 28, no. 2, pp. 129–137, 1982.

[9] J. MacQueen, "Some methods for classification and analysis of multivariate observations," in *Proc. 5th Berkeley Symp. Mathematical Statistics and Probability*, vol. 1, 1967, pp. 281–297.

[10] B. Bahmani, B. Moseley, A. Vattani, R. Kumar, and S. Vassilvitskii, "Scalable k-means++," *Proc. VLDB Endowment*, vol. 5, no. 7, pp. 622–633, 2012.

[11] M. Armbrust *et al.*, "Spark SQL: Relational data processing in Spark," in *Proc. 2015 ACM SIGMOD Int. Conf. Management of Data*, Melbourne, Australia, 2015, pp. 1383–1394.

[12] The Apache Software Foundation, "Cluster mode overview," *Apache Spark Documentation*. [Online]. Available: https://spark.apache.org/docs/latest/cluster-overview.html (accedido: nov. 2026).

[13] The Apache Software Foundation, "Clustering — K-means," *Apache Spark MLlib Guide*. [Online]. Available: https://spark.apache.org/docs/latest/ml-clustering.html (accedido: nov. 2026).

[14] The Apache Software Foundation, "Spark SQL, DataFrames and Datasets Guide," *Apache Spark Documentation*. [Online]. Available: https://spark.apache.org/docs/latest/sql-programming-guide.html (accedido: nov. 2026).

[15] The Apache Software Foundation, "Spark configuration," *Apache Spark Documentation*. [Online]. Available: https://spark.apache.org/docs/latest/configuration.html (accedido: nov. 2026).

[16] The Apache Software Foundation, "Tuning Spark," *Apache Spark Documentation*. [Online]. Available: https://spark.apache.org/docs/latest/tuning.html (accedido: nov. 2026).

[17] The PostgreSQL Global Development Group, "Transactions," *PostgreSQL Documentation*. [Online]. Available: https://www.postgresql.org/docs/current/tutorial-transactions.html (accedido: nov. 2026).